Finetuning a Sentence Transformer model on Named Entity Recognition

This notebook uses this guide and tutorial as a reference:
https://huggingface.co/docs/transformers/en/tasks/token_classification

In [1]:
```
%%capture
!pip install -U sentence-transformers
!pip install datasets==3.6.0
!pip install transformers
!pip install evaluate
!pip install seqeval
```

In [2]:
```
import numpy as np
# Huggingface imports below
from datasets import load_dataset, DatasetDict
from transformers import AutoTokenizer, DataCollatorWithPadding
from transformers import AutoModelForSequenceClassification, TrainingArguments, Tra
from transformers import DataCollatorForTokenClassification
from transformers import AutoModelForTokenClassification, AutoTokenizer, TrainingAr
from transformers import pipeline
import evaluate
```

In [3]:
```
from sentence_transformers import SentenceTransformer

# Here I am initializing a pretrained SentenceTransformer model
sbert_model = SentenceTransformer("all-mpnet-base-v2")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarni
ng:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (http
s://huggingface.co/settings/tokens), set it as secret in your Google Colab and resta
rt your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public m
odels or datasets.
  warnings.warn(
```

Task A: Named Entity Recognition

In this cell I am establishing the Named Entity Recognition dataset for this exercise, which is the MIT Movie Trivia Dataset.

In [4]:
```
movie_trivia_ds = load_dataset("tner/mit_movie_trivia")

movie_trivia_train_ds = movie_trivia_ds["train"].to_pandas()
movie_trivia_validation_ds = movie_trivia_ds["validation"].to_pandas()
movie_trivia_test_ds = movie_trivia_ds["test"].to_pandas()
```

```
print(movie_trivia_ds)
print(movie_trivia_train_ds.iloc[0])
```

```
DatasetDict({
    train: Dataset({
        features: ['tokens', 'tags'],
        num_rows: 6816
    })
    validation: Dataset({
        features: ['tokens', 'tags'],
        num_rows: 1000
    })
    test: Dataset({
        features: ['tokens', 'tags'],
        num_rows: 1953
    })
})
tokens    [what, 1995, romantic, comedy, film, starred, ...
tags      [0, 9, 10, 15, 0, 0, 1, 2, 0, 3, 4, 4, 4, 4, 4...
Name: 0, dtype: object
```

The dataset, for each split, contains tokens of a sentence and numeric labels for each token. These tokens are in the Beginning, Inside, Outside, or BIO, notation. Below I define dictionary objects that connect the numeric labels to their text equivalents.

In [5]:
```
id2label = {
    0: "O",
    1: "B-Actor",
    2: "I-Actor",
    3: "B-Plot",
    4: "I-Plot",
    5: "B-Opinion",
    6: "I-Opinion",
    7: "B-Award",
    8: "I-Award",
    9: "B-Year",
    10: "B-Genre",
    11: "B-Origin",
    12: "I-Origin",
    13: "B-Director",
    14: "I-Director",
    15: "I-Genre",
    16: "I-Year",
    17: "B-Soundtrack",
    18: "I-Soundtrack",
    19: "B-Relationship",
    20: "I-Relationship",
    21: "B-Character_Name",
    22: "I-Character_Name",
    23: "B-Quote",
    24: "I-Quote"
}

label2id = {
```

```
    "O": 0,
    "B-Actor": 1,
    "I-Actor": 2,
    "B-Plot": 3,
    "I-Plot": 4,
    "B-Opinion": 5,
    "I-Opinion": 6,
    "B-Award": 7,
    "I-Award": 8,
    "B-Year": 9,
    "B-Genre": 10,
    "B-Origin": 11,
    "I-Origin": 12,
    "B-Director": 13,
    "I-Director": 14,
    "I-Genre": 15,
    "I-Year": 16,
    "B-Soundtrack": 17,
    "I-Soundtrack": 18,
    "B-Relationship": 19,
    "I-Relationship": 20,
    "B-Character_Name": 21,
    "I-Character_Name": 22,
    "B-Quote": 23,
    "I-Quote": 24
}
```

In this code segment below, I am writing out a sample from the 'train' split of the dataset on one line along with the respective BIO tags on the next line.

The tokens and tags are aligned with each other to show how these tags define how each token is meant to be classified and trained towards.

In [6]:
```
print(movie_trivia_ds["train"][0])
words = movie_trivia_ds["train"][0]["tokens"]
labels = movie_trivia_ds["train"][0]["tags"]
line1 = ""
line2 = ""
for word, label in zip(words, labels):
    full_label = id2label[label]
    max_length = max(len(word), len(full_label))
    line1 += word + " " * (max_length - len(word) + 1)
    line2 += full_label + " " * (max_length - len(full_label) + 1)

print(line1)
print(line2)
```

```
{'tokens': ['what', '1995', 'romantic', 'comedy', 'film', 'starred', 'michael', 'dou
glas', 'as', 'a', 'u', 's', 'head', 'of', 'state', 'looking', 'for', 'love'], 'tag
s': [0, 9, 10, 15, 0, 0, 1, 2, 0, 3, 4, 4, 4, 4, 4, 4, 4, 4]}
what 1995   romantic comedy  film starred michael douglas as a      u      s      he
ad   of     state  looking for    love
O    B-Year B-Genre  I-Genre O    O       B-Actor I-Actor O  B-Plot I-Plot I-Plot I-
Plot I-Plot I-Plot I-Plot  I-Plot I-Plot
```

In this cell I am establishing the tokenizer to use for our training loop using the all-mpnet-base-v2 sentence transformer from the sentence-transformer library.

The code essentially prints a sample of tokens from the 'train' dataset, processes those tokens using the tokenizer, and then converts the output ids back to the tokens.

In [7]:
```
# establish tokenizer
ner_tokenizer = AutoTokenizer.from_pretrained("sentence-transformers/all-mpnet-base

print(movie_trivia_train_ds.iloc[0]["tokens"])

# generate embeddings for one input
inputs = ner_tokenizer(list(movie_trivia_train_ds.iloc[0]["tokens"]), is_split_into
print(inputs)
tokens = ner_tokenizer.convert_ids_to_tokens(inputs["input_ids"])
print(tokens)
```

```
['what' '1995' 'romantic' 'comedy' 'film' 'starred' 'michael' 'douglas'
 'as' 'a' 'u' 's' 'head' 'of' 'state' 'looking' 'for' 'love']
{'input_ids': [0, 2058, 2790, 6302, 4042, 2147, 5656, 2749, 5207, 2008, 1041, 1061,
1059, 2136, 2001, 2114, 2563, 2009, 2297, 2], 'attention_mask': [1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]}
['<s>', 'what', '1995', 'romantic', 'comedy', 'film', 'starred', 'michael', 'dougla
s', 'as', 'a', 'u', 's', 'head', 'of', 'state', 'looking', 'for', 'love', '</s>']
```

The above cell also displays how special tokens get added by the tokenizer's process, such as the [CLS], [SEP], or ~~tokens, and subword tokenization could create a mismatch between the input and labels.~~

This cell is a preprocessing loop that realigns the tokens and labels by:

1. Mapping all tokens to their corresponding word with the word_ids method.
2. Assigning the label -100 to the special tokens [CLS] and [SEP] so they're ignored by the PyTorch loss function (see CrossEntropyLoss).
3. Only labeling the first token of a given word. Assign -100 to other subtokens from the same word.

In [8]:
```
# preprocessing loop
def align_labels_with_tokens(labels, word_ids):
    new_labels = []
    current_word = None
    for word_id in word_ids:
        if word_id != current_word:
            # Start of a new word!
            current_word = word_id
            label = -100 if word_id is None else labels[word_id]
            new_labels.append(label)
        elif word_id is None:
            # Special token
            new_labels.append(-100)
        else:
            # Same word as previous token
```

```
            label = labels[word_id]
            # If the label is B-XXX we change it to I-XXX
            if label % 2 == 1:
                label += 1
            new_labels.append(label)

    return new_labels


# test the preprocessing loop
labels = movie_trivia_ds["train"][0]["tags"]
word_ids = inputs.word_ids()
print(labels)
print(align_labels_with_tokens(labels, word_ids))

def tokenize_and_align_labels(examples):
    tokenized_inputs = ner_tokenizer(
        examples["tokens"], truncation=True, is_split_into_words=True
    )
    all_labels = examples["tags"]
    new_labels = []
    for i, labels in enumerate(all_labels):
        word_ids = tokenized_inputs.word_ids(i)
        new_labels.append(align_labels_with_tokens(labels, word_ids))

    tokenized_inputs["labels"] = new_labels
    return tokenized_inputs

tokenized_movie_trivia_datasets = movie_trivia_ds.map(
    tokenize_and_align_labels,
    batched=True,
    remove_columns=movie_trivia_ds["train"].column_names,
)

print(tokenized_movie_trivia_datasets["train"][0:5])
```

```
[0, 9, 10, 15, 0, 0, 1, 2, 0, 3, 4, 4, 4, 4, 4, 4, 4, 4]
[-100, 0, 9, 10, 15, 0, 0, 1, 2, 0, 3, 4, 4, 4, 4, 4, 4, 4, 4, -100]
Map:   0%|          | 0/1953 [00:00<?, ? examples/s]
```

```
{'input_ids': [[0, 2058, 2790, 6302, 4042, 2147, 5656, 2749, 5207, 2008, 1041, 1061,
1059, 2136, 2001, 2114, 2563, 2009, 2297, 2], [0, 2058, 3855, 2333, 2147, 4630, 285
2, 19045, 2005, 2245, 2010, 2000, 2342, 2001, 2000, 2172, 2175, 2015, 7727, 15540, 2
9329, 3055, 2], [0, 4922, 23335, 2860, 2002, 5656, 2003, 2027, 4442, 4535, 2000, 221
4, 12521, 2365, 2003, 2000, 19902, 2002, 5987, 1041, 10822, 2], [0, 2058, 2234, 404
2, 2519, 3893, 2733, 3367, 2002, 2707, 25302, 2736, 2003, 2077, 2733, 3367, 23937, 2
5302, 1059, 3241, 2287, 2], [0, 2058, 2007, 2000, 2147, 2059, 2000, 2201, 2158, 200
1, 2000, 2099, 2012, 4080, 2000, 2170, 2001, 2371, 2115, 2010, 2012, 2158, 2]], 'att
ention_mask': [[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1], [1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1], [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1]], 'labels': [[-100, 0, 9, 10, 15, 0, 0, 1, 2, 0, 3, 4, 4, 4, 4,
4, 4, 4, 4, -100], [-100, 0, 9, 10, 0, 0, 1, 2, 0, 11, 12, 12, 12, 12, 12, 12, 12, 1
2, 12, 12, 12, 12, -100], [-100, 13, 14, 0, 0, 0, 0, 0, 5, 0, 3, 4, 4, 4, 4, 4, 4,
4, 4, 4, 4, 4, -100], [-100, 0, 9, 10, 0, 1, 2, 2, 0, 1, 2, 0, 0, 0, 1, 2, 3, 1, 0, 3,
4, -100], [-100, 0, 0, 0, 0, 0, 3, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, -10
0]]}
```

In the next two cells a data collator is defined, which will pad the sentences in a batch to the length of the longest sentence in the batch. This is necessary for the artificial intelligence model to accept the inputs as a uniform group.

In [9]:
```
# establish a data collator
data_collator = DataCollatorForTokenClassification(tokenizer=ner_tokenizer)
```

In [10]:
```
collated_movie_batch = data_collator([tokenized_movie_trivia_datasets["train"][i] f
collated_movie_batch["labels"]
```

Out[10]:
```
tensor([[-100,    0,    9,   10,   15,    0,    0,    1,    2,    0,    3,    4,
             4,    4,    4,    4,    4,    4, -100, -100, -100, -100],
        [-100,    0,    9,   10,    0,    0,    1,    2,    0,   11,   12,   12,
            12,   12,   12,   12,   12,   12,   12,   12,   12, -100]])
```

This cell displays a few samples from the training dataset.

In [11]:
```
for i in range(2):
    print(tokenized_movie_trivia_datasets["train"][i]["labels"])

for i in range(2):
    print(movie_trivia_ds["train"][i]["tokens"])
    labelId2textValue = []
    for label in tokenized_movie_trivia_datasets["train"][i]["labels"]:
      if label != -100:
        labelId2textValue.append(id2label[label])
    print(labelId2textValue)
```

```
[-100, 0, 9, 10, 15, 0, 0, 1, 2, 0, 3, 4, 4, 4, 4, 4, 4, 4, 4, -100]
[-100, 0, 9, 10, 0, 0, 1, 2, 0, 11, 12, 12, 12, 12, 12, 12, 12, 12, 12, 12, 12,
-100]
['what', '1995', 'romantic', 'comedy', 'film', 'starred', 'michael', 'douglas', 'a
s', 'a', 'u', 's', 'head', 'of', 'state', 'looking', 'for', 'love']
['O', 'B-Year', 'B-Genre', 'I-Genre', 'O', 'O', 'B-Actor', 'I-Actor', 'O', 'B-Plot',
'I-Plot', 'I-Plot', 'I-Plot', 'I-Plot', 'I-Plot', 'I-Plot', 'I-Plot', 'I-Plot']
['what', '1959', 'british', 'film', 'starring', 'peter', 'sellers', 'was', 'based',
'on', 'the', 'book', 'of', 'the', 'same', 'name', 'by', 'leonard', 'wibberley']
['O', 'B-Year', 'B-Genre', 'O', 'O', 'B-Actor', 'I-Actor', 'O', 'B-Origin', 'I-Origi
n', 'I-Origin', 'I-Origin', 'I-Origin', 'I-Origin', 'I-Origin', 'I-Origin', 'I-Origi
n', 'I-Origin', 'I-Origin', 'I-Origin', 'I-Origin']
```

This cell defines the 'compute_metrics' function that will be used by the training loop to both train the model and inform the user in what direction the training is going.

In [12]:
```python
# establish metrics
import evaluate

metric = evaluate.load("seqeval")

ner_feature = movie_trivia_ds["train"].features["tags"]
label_names = []
for value in id2label.values():
  label_names.append(value)

print(label_names)

def compute_metrics(eval_preds):
    logits, labels = eval_preds
    predictions = np.argmax(logits, axis=-1)

    # Remove ignored index (special tokens) and convert to labels
    true_labels = [[label_names[l] for l in label if l != -100] for label in labels
    true_predictions = [
        [label_names[p] for (p, l) in zip(prediction, label) if l != -100]
        for prediction, label in zip(predictions, labels)
    ]
    all_metrics = metric.compute(predictions=true_predictions, references=true_labe
    return {
        "precision": all_metrics["overall_precision"],
        "recall": all_metrics["overall_recall"],
        "f1": all_metrics["overall_f1"],
        "accuracy": all_metrics["overall_accuracy"],
    }
```

```
['O', 'B-Actor', 'I-Actor', 'B-Plot', 'I-Plot', 'B-Opinion', 'I-Opinion', 'B-Award',
'I-Award', 'B-Year', 'B-Genre', 'B-Origin', 'I-Origin', 'B-Director', 'I-Director',
'I-Genre', 'I-Year', 'B-Soundtrack', 'I-Soundtrack', 'B-Relationship', 'I-Relationsh
ip', 'B-Character_Name', 'I-Character_Name', 'B-Quote', 'I-Quote']
```

The Token Classification version of the all-mpnet-base-v2 sentence transformer model is defined here that will be finetuned. The id to label maps are provided as inputs for this model initialization.

In [13]:
```
# now we initalize our sentence transformer model
ner_model = AutoModelForTokenClassification.from_pretrained(
    "sentence-transformers/all-mpnet-base-v2", num_labels=25, id2label=id2label, la
)
```

```
Some weights of MPNetForTokenClassification were not initialized from the model chec
kpoint at sentence-transformers/all-mpnet-base-v2 and are newly initialized: ['class
ifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for
predictions and inference.
```

Here the TrainingArguments are defined and the Trainer class is run to finetune the model we just initialized, while using the data_collator and ner_tokenizer we previously defined.

In [14]:
```
from transformers import TrainingArguments
from transformers import Trainer

import os
os.environ['CUDA_LAUNCH_BLOCKING'] = '1'

# we train a model
args = TrainingArguments(
    "fine-tuned-all-mpnet-base-v2",
    eval_strategy="epoch",
    save_strategy="best",
    metric_for_best_model="f1",
    learning_rate=2e-5,
    num_train_epochs=8,
    weight_decay=0.01,
    report_to=["none"],
    push_to_hub=False,
)


trainer = Trainer(
    model=ner_model,
    args=args,
    train_dataset=tokenized_movie_trivia_datasets["train"],
    eval_dataset=tokenized_movie_trivia_datasets["validation"],
    data_collator=data_collator,
    compute_metrics=compute_metrics,
    tokenizer=ner_tokenizer,
)
trainer.train()
```

```
/tmp/ipython-input-3613791060.py:21: FutureWarning: `tokenizer` is deprecated and wi
ll be removed in version 5.0.0 for `Trainer.__init__`. Use `processing_class` instea
d.
  trainer = Trainer(
```

[6816/6816 11:56, Epoch 8/8]

Epoch	Training Loss	Validation Loss	Precision	Recall	F1	Accuracy
1	1.530600	0.713002	0.638664	0.658333	0.648350	0.875816
2	0.568600	0.492577	0.640295	0.692029	0.665158	0.882388
3	0.399400	0.417466	0.678106	0.715942	0.696510	0.889420
4	0.327900	0.405144	0.687759	0.720652	0.703822	0.891212
5	0.269900	0.382636	0.681135	0.722101	0.701020	0.891396
6	0.234300	0.400343	0.674890	0.723551	0.698374	0.891626
7	0.216300	0.403126	0.676391	0.722464	0.698669	0.889282
8	0.191700	0.405551	0.678064	0.725725	0.701085	0.889972

```
/usr/local/lib/python3.12/dist-packages/seqeval/metrics/v1.py:57: UndefinedMetricWar
ning: Precision and F-score are ill-defined and being set to 0.0 in labels with no p
redicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
/usr/local/lib/python3.12/dist-packages/seqeval/metrics/v1.py:57: UndefinedMetricWar
ning: Precision and F-score are ill-defined and being set to 0.0 in labels with no p
redicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
/usr/local/lib/python3.12/dist-packages/seqeval/metrics/v1.py:57: UndefinedMetricWar
ning: Precision and F-score are ill-defined and being set to 0.0 in labels with no p
redicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
/usr/local/lib/python3.12/dist-packages/seqeval/metrics/v1.py:57: UndefinedMetricWar
ning: Precision and F-score are ill-defined and being set to 0.0 in labels with no p
redicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
/usr/local/lib/python3.12/dist-packages/seqeval/metrics/v1.py:57: UndefinedMetricWar
ning: Precision and F-score are ill-defined and being set to 0.0 in labels with no p
redicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
```

Out[14]:
```
TrainOutput(global_step=6816, training_loss=0.43323402617459006, metrics={'train_r
untime': 717.8661, 'train_samples_per_second': 75.958, 'train_steps_per_second':
9.495, 'total_flos': 938299420737600.0, 'train_loss': 0.43323402617459006, 'epoc
h': 8.0})
```

This cell finds the 'best' checkpoint saved by the 'Trainer' object. The Trainer object's 'save_strategy' is set to 'best' which only saves a training checkpoint if the most recent checkpoint has an improvement in metrics over the previous saves.

The best checkpoint is the checkpoint with the highest numeric suffix, or the most recent checkpoint. This code finds all the checkpoint names in the save directory, gets their numeric suffixes and returns the directory name that has the highest numeric suffix.

In [15]:
```
saved_checkpoints = os.listdir("./fine-tuned-all-mpnet-base-v2/")
print(saved_checkpoints)
```

```
import re

def get_numeric_suffix(s):
    """
    Extracts the numeric suffix from a string and converts it to an integer.
    Returns 0 if no numeric suffix is found.
    """
    # Regex to find one or more digits (\d+) at the end ($) of the string
    match = re.search(r"(\d+)$", s)
    if match:
        return int(match.group(1)) # Convert the matched string to an integer
    return 0

def find_string_with_largest_suffix(string_list):
    """
    Finds the string in a list with the largest numeric suffix.
    """
    if not string_list:
        return None

    # Use the max function with a key that returns the numeric suffix for compariso
    largest_string = max(string_list, key=get_numeric_suffix)
    return largest_string

best_checkpoint_name = find_string_with_largest_suffix(saved_checkpoints)
print(best_checkpoint_name)
```

```
['checkpoint-3408', 'checkpoint-1704', 'checkpoint-852', 'checkpoint-2556']
checkpoint-3408
```

In these cells, we load a finetuned model from a checkpoint and apply that model on data from the 'validation' split of the data. In the second cell some postprocessing is applied that makes the outputs easier to interpret and display.

In [16]:
```
ner_model_checkpoint = AutoModelForTokenClassification.from_pretrained(
    "./fine-tuned-all-mpnet-base-v2/" + best_checkpoint_name, num_labels=25, id2lab
)

texts = movie_trivia_ds["validation"][1:6]["tokens"]
print(texts)
classifier = pipeline("ner", model=ner_model, tokenizer=ner_tokenizer)

for text in texts:
  print(classifier(text))
```

```
Device set to use cuda:0
```

```
[['liza', 'minnelli', 'and', 'joel', 'gray', 'won', 'oscars', 'for', 'their', 'role
s', 'in', 'this', '1972', 'movie', 'that', 'follows', 'nightclub', 'entertainers',
'in', 'berlin', 'as', 'the', 'nazis', 'come', 'to', 'power'], ['what', 'is', 'that',
'tom', 'hanks', 'and', 'julia', 'roberts', 'movie', 'about', 'hanks', 'who', 'play
s', 'a', 'down', 'on', 'his', 'luck', 'average', 'guy', 'who', 'goes', 'back', 'to',
'college', 'and', 'gets', 'taught', 'by', 'roberts'], ['what', 'is', 'the', 'movie',
'making', 'fun', 'of', 'macgyver', 'by', 're', 'enacting', 'scenes', 'similar', 't
o', 'his', 'movies'], ['i', 'am', 'thinking', 'of', 'an', 'animated', 'film', 'base
d', 'on', 'a', 'classic', 'theodor', 'geisel', 'children', 's', 'novel', 'about',
'a', 'young', 'boy', 's', 'quest', 'to', 'save', 'the', 'trees'], ['what', '1981',
'feature', 'film', 'starring', 'mel', 'gibson', 'takes', 'place', 'in', 'a', 'post',
'apocalyptic', 'world', 'in', 'australia']]
[[{'entity': 'B-Character_Name', 'score': np.float32(0.6758772), 'index': 1, 'word':
'liza', 'start': 0, 'end': 4}], [{'entity': 'B-Character_Name', 'score': np.float32
(0.44294918), 'index': 1, 'word': 'min', 'start': 0, 'end': 3}], [], [{'entity': 'B-
Character_Name', 'score': np.float32(0.65728885), 'index': 1, 'word': 'joel', 'star
t': 0, 'end': 4}], [{'entity': 'B-Character_Name', 'score': np.float32(0.21760172),
'index': 1, 'word': 'gray', 'start': 0, 'end': 4}], [], [{'entity': 'B-Award', 'scor
e': np.float32(0.58952624), 'index': 1, 'word': 'oscar', 'start': 0, 'end': 5}, {'en
tity': 'I-Award', 'score': np.float32(0.6389345), 'index': 2, 'word': '##s', 'star
t': 5, 'end': 6}], [], [], [], [], [], [{'entity': 'B-Year', 'score': np.float32(0.9
508221), 'index': 1, 'word': '1972', 'start': 0, 'end': 4}], [], [], [], [], [{'enti
ty': 'B-Plot', 'score': np.float32(0.58476883), 'index': 1, 'word': 'entertainer',
'start': 0, 'end': 11}], [], [], [], [], [], [], [], []]
[[], [], [], [{'entity': 'B-Character_Name', 'score': np.float32(0.70006084), 'inde
x': 1, 'word': 'tom', 'start': 0, 'end': 3}], [{'entity': 'B-Actor', 'score': np.flo
at32(0.9568042), 'index': 1, 'word': 'hank', 'start': 0, 'end': 4}, {'entity': 'I-Ac
tor', 'score': np.float32(0.9756929), 'index': 2, 'word': '##s', 'start': 4, 'end':
5}], [], [{'entity': 'B-Character_Name', 'score': np.float32(0.5821569), 'index': 1,
'word': 'julia', 'start': 0, 'end': 5}], [{'entity': 'B-Character_Name', 'score': n
p.float32(0.39328784), 'index': 1, 'word': 'roberts', 'start': 0, 'end': 7}], [],
[], [{'entity': 'B-Actor', 'score': np.float32(0.9568042), 'index': 1, 'word': 'han
k', 'start': 0, 'end': 4}, {'entity': 'I-Actor', 'score': np.float32(0.9756929), 'in
dex': 2, 'word': '##s', 'start': 4, 'end': 5}], [], [], [], [], [], [], [], [{'entit
y': 'B-Plot', 'score': np.float32(0.11628301), 'index': 1, 'word': 'average', 'star
t': 0, 'end': 7}], [{'entity': 'B-Plot', 'score': np.float32(0.203401), 'index': 1,
'word': 'guy', 'start': 0, 'end': 3}], [], [], [], [], [], [], [], [], [], [{'entit
y': 'B-Character_Name', 'score': np.float32(0.39328784), 'index': 1, 'word': 'robert
s', 'start': 0, 'end': 7}]]
[[], [], [], [], [], [{'entity': 'B-Genre', 'score': np.float32(0.27718678), 'inde
x': 1, 'word': 'fun', 'start': 0, 'end': 3}], [], [], [], [], [{'entity': 'B-Plot',
'score': np.float32(0.48454943), 'index': 1, 'word': 'en', 'start': 0, 'end': 2},
{'entity': 'I-Plot', 'score': np.float32(0.64145756), 'index': 2, 'word': '##act',
'start': 2, 'end': 5}], [], [], [], [], []]
[[], [], [], [], [], [{'entity': 'B-Genre', 'score': np.float32(0.8832682), 'index':
1, 'word': 'animated', 'start': 0, 'end': 8}], [], [], [], [], [{'entity': 'B-Opinio
n', 'score': np.float32(0.57827765), 'index': 1, 'word': 'classic', 'start': 0, 'en
d': 7}], [{'entity': 'B-Character_Name', 'score': np.float32(0.7119594), 'index': 1,
'word': 'theodor', 'start': 0, 'end': 7}], [{'entity': 'B-Character_Name', 'score':
np.float32(0.4045935), 'index': 1, 'word': 'ge', 'start': 0, 'end': 2}, {'entity':
'I-Character_Name', 'score': np.float32(0.2684256), 'index': 2, 'word': '##ise', 'st
art': 2, 'end': 5}], [], [], [{'entity': 'B-Genre', 'score': np.float32(0.5211752),
'index': 1, 'word': 'novel', 'start': 0, 'end': 5}], [], [], [], [], [], [{'entity':
'B-Plot', 'score': np.float32(0.20749801), 'index': 1, 'word': 'quest', 'start': 0,
'end': 5}], [], [], [], []]
[[], [{'entity': 'B-Year', 'score': np.float32(0.95317143), 'index': 1, 'word': '198
```

```
1', 'start': 0, 'end': 4}], [], [], [{'entity': 'B-Opinion', 'score': np.float32(0.2
3297721), 'index': 1, 'word': 'starring', 'start': 0, 'end': 8}], [{'entity': 'B-Cha
racter_Name', 'score': np.float32(0.67288566), 'index': 1, 'word': 'mel', 'start':
0, 'end': 3}], [{'entity': 'B-Character_Name', 'score': np.float32(0.33859286), 'ind
ex': 1, 'word': 'gibson', 'start': 0, 'end': 6}], [], [], [], [], [], [{'entity': 'B
-Genre', 'score': np.float32(0.5631824), 'index': 1, 'word': 'apocalyptic', 'start':
0, 'end': 11}], [], [], []]
```

In [17]:
```
texts = movie_trivia_ds["validation"][1:6]["tokens"]
classifier = pipeline("token-classification", model=ner_model_checkpoint, tokenizer

predictions = []
for text in texts:
  text_as_sentence = " ".join(text)

  classifier_output = classifier(text_as_sentence)
  named_entities = []

  for prediction in classifier_output:
    named_entities.append({
        "word": prediction["word"],
        "entity_group": prediction["entity_group"]
    })

  prediction = {
      "input_text": text_as_sentence,
      "predictions": named_entities
  }
  predictions.append(prediction)

for prediction in predictions:
  print(prediction)
```

```
Device set to use cuda:0
```

{'input_text': 'liza minnelli and joel gray won oscars for their roles in this 1972 movie that follows nightclub entertainers in berlin as the nazis come to power', 'predictions': [{'word': 'liza minnelli', 'entity_group': 'Actor'}, {'word': 'joel gray', 'entity_group': 'Actor'}, {'word': 'oscars', 'entity_group': 'Award'}, {'word': '1972', 'entity_group': 'Year'}, {'word': 'follows', 'entity_group': 'Plot'}, {'word': 'nightclub entertainers in berlin as the nazis come to power', 'entity_group': 'Plot'}]}
{'input_text': 'what is that tom hanks and julia roberts movie about hanks who plays a down on his luck average guy who goes back to college and gets taught by roberts', 'predictions': [{'word': 'tom hanks', 'entity_group': 'Actor'}, {'word': 'julia roberts', 'entity_group': 'Actor'}, {'word': 'hanks', 'entity_group': 'Actor'}, {'word': 'plays a down on his luck average guy who goes back to college and gets taught by roberts', 'entity_group': 'Plot'}]}
{'input_text': 'what is the movie making fun of macgyver by re enacting scenes similar to his movies', 'predictions': [{'word': 'making fun of macgyver by re enacting scenes similar to his movies', 'entity_group': 'Plot'}]}
{'input_text': 'i am thinking of an animated film based on a classic theodor geisel children s novel about a young boy s quest to save the trees', 'predictions': [{'word': 'animated', 'entity_group': 'Genre'}, {'word': 'classic', 'entity_group': 'Opinion'}, {'word': 'theodor geisel children s novel', 'entity_group': 'Origin'}, {'word': 'a young boy s quest to save the trees', 'entity_group': 'Plot'}]}
{'input_text': 'what 1981 feature film starring mel gibson takes place in a post apocalyptic world in australia', 'predictions': [{'word': '1981', 'entity_group': 'Year'}, {'word': 'mel gibson', 'entity_group': 'Actor'}, {'word': 'takes place in a post apocalyptic world in australia', 'entity_group': 'Plot'}]}